

Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Elaborazione di dati in un array

Il nostro esperimento: il periodo di un pendolo

Pendolo con $L = 1.000$ m: **100 misure** del periodo T con una fotocellula (risoluzione 1 ms).

```
$ head -4 periodi.txt
1.994
2.010
1.978
2.027
$ wc -l periodi.txt
     100 periodi.txt
```

I dati sono **inventati** (simulati al computer), ma con le caratteristiche di misure vere. Il file è tra i materiali della lezione.

- Le misure **fluttuano**: errori casuali (attrito, avvio della fotocellula, …).
- Domande tipiche di laboratorio:
  - qual è il valore **medio** di T?
  - quanto sono **disperse** le misure?
  - qual è l'**incertezza** sul valore medio?
  - che **forma** ha la distribuzione?
- Teoria: $T = 2\pi\sqrt{L/g}$, da cui si ricava g.
- Tutti i programmi leggono i dati con `./prog < periodi.txt` (lezione 17).

La media

$$\bar{T} = \frac{1}{N} \sum_{i=0}^{N-1} T_i$$

```c
#include <stdio.h>

#define N 100

int main() {
    double T[N];              // periodi [s]
    double somma = 0.0;
    int i;

    for (i = 0; i < N; i++) {
        scanf("%lf", &T[i]);
    }
    for (i = 0; i < N; i++) {
        somma += T[i];
    }
    double media = somma / N;
    printf("media = %.4f s\n", media);
    return 0;
}
```

```
$ gcc -Wall -Wextra -o media media.c
$ ./media < periodi.txt
media = 2.0067 s
```

- `somma` è un **accumulatore** (lezione 15): va inizializzato a **0**.
- La sommatoria $\sum_{i=0}^{N-1}$ diventa il ciclo `for` su tutti gli indici.
- `somma / N`: `somma` è `double`, quindi la divisione è reale anche se `N` è intero.
- **Test:** con i dati `{1, 2, 3}` il programma deve dare `2`.

Minimo e massimo

```c
double min = T[0], max = T[0];
int imin = 0, imax = 0;
for (i = 1; i < N; i++) {
    if (T[i] < min) {
        min = T[i];
        imin = i;
    }
    if (T[i] > max) {
        max = T[i];
        imax = i;
    }
}
printf("min = %.3f s (misura %d)\n", min, imin);
printf("max = %.3f s (misura %d)\n", max, imax);
```

```
$ ./minmax < periodi.txt
min = 1.977 s (misura 91)
max = 2.053 s (misura 49)
```

- Idea: tenere il "**campione** provvisorio" e confrontarlo con ogni elemento.
- Si parte dal **primo elemento**, non da 0: se tutti i dati fossero positivi, `min = 0` non verrebbe mai superato!
- Il ciclo può partire da `i = 1`: `T[0]` è già stato considerato.
- Salvando anche l'**indice** sappiamo **quale** misura è la più piccola: utile per trovare misure anomale.
- Con `<` stretto, a parità di valore resta il **primo** trovato.

Varianza e deviazione standard

Quanto sono **disperse** le misure attorno alla media?

Varianza campionaria

$$s^2 = \frac{1}{N-1} \sum_{i=0}^{N-1} \left(T_i - \bar{T}\right)^2$$

Deviazione standard e **errore sulla media**

$$\sigma = \sqrt{s^2} \qquad \sigma_{\bar{T}} = \frac{\sigma}{\sqrt{N}}$$

- σ è l'incertezza sulla **singola** misura: ha le stesse unità di T.
- $\sigma_{\bar{T}}$ è l'incertezza sulla **media**: diminuisce come $1/\sqrt{N}$.
- $N-1$ e non N: la media è stata stimata dagli stessi dati (lo vedrete nei corsi di laboratorio).
- Per calcolare s^2 serve **già** conoscere $\bar{T}$: bisogna scorrere l'array **due volte**.
- Ecco perché conviene **memorizzare** i dati in un array!

Varianza: la formula a due passate

```c
double media = 0.0, var = 0.0;

for (i = 0; i < N; i++) {          // prima passata
    media += T[i];
}
media /= N;
for (i = 0; i < N; i++) {          // seconda passata
    var += (T[i] - media) * (T[i] - media);
}
var /= (N - 1);
double sigma = sqrt(var);
double err   = sigma / sqrt(N);

printf("media = %.4f s\n", media);
printf("sigma = %.4f s\n", sigma);
printf("T     = %.4f +- %.4f s\n", media, err);
```

Il resto del programma (`stat.c`) è come `media.c` , con in più `#include <math.h>` .

```
$ gcc -Wall -Wextra -o stat stat.c -lm
$ ./stat < periodi.txt
media = 2.0067 s
sigma = 0.0153 s
T     = 2.0067 +- 0.0015 s
```

- `sqrt` viene da `math.h` : serve `-lm` (lezione 13).
- Il quadrato si scrive come prodotto: niente `^` !
- `N - 1` è intero, ma `var` è `double` : divisione reale.
- $\sigma \approx 15$ ms, ma $\sigma_{\bar{T}} \approx 1.5$ ms: la media è $\sqrt{100} = 10$ volte più precisa.

Varianza: la formula a una passata

Sviluppando il quadrato si ottiene una formula che richiede **un solo** ciclo:

$$s^2 = \frac{N}{N-1}\left(\overline{T^2} - \bar{T}^2\right) \qquad \overline{T^2} = \frac{1}{N}\sum T_i^2$$

```
1   double s1 = 0.0, s2 = 0.0;
2
3   for (i = 0; i < N; i++) {       // una sola passata
4       s1 += T[i];
5       s2 += T[i] * T[i];
6   }
7   double media = s1 / N;
8   double var   = (s2 / N - media * media) * N / (N - 1);
```

```
1   $ ./unapassata < periodi.txt
2   media = 2.0067 s
3   sigma = 0.0153 s
```

- Stesso risultato, e i dati si possono leggere ed elaborare "al volo", **senza** array.
- Matematicamente le due formule sono **identiche**.
- Numericamente **no**: $\overline{T^2}$ e $\bar{T}^2$ sono due numeri grandi e quasi uguali, e li stiamo **sottraendo**…

È la **cancellazione** vista nella lezione 4: sottraendo numeri vicini si perdono cifre significative.

Quando la formula a una passata fallisce

Un oscillatore al quarzo da orologio ha frequenza nominale 32768 Hz. Ne misuriamo 5 volte la frequenza:

```c
#include <stdio.h>
#define N 5

int main() {
    double f[N] = {32768.00012, 32768.00010, 32768.00013,
                   32768.00011, 32768.00014};   // [Hz]
    double s1 = 0.0, s2 = 0.0, var2 = 0.0;
    int i;
    for (i = 0; i < N; i++) {
        s1 += f[i];
        s2 += f[i] * f[i];
    }
    double media = s1 / N;
    double var1  = (s2 / N - media * media) * N / (N - 1);
    for (i = 0; i < N; i++) {
        var2 += (f[i] - media) * (f[i] - media);
    }
    var2 /= (N - 1);
    printf("una passata:  var = %e Hz^2\n", var1);
    printf("due passate:  var = %e Hz^2\n", var2);
    return 0;
}
```

```
$ ./quarzo
una passata:  var = 2.800232e-07 Hz^2
due passate:  var = 2.500000e-10 Hz^2
```

- Valore esatto: 2.5×10^{-10} Hz². Una passata sbaglia di un fattore **mille**!
- $\overline{f^2} \approx 10^9$ e cerchiamo una differenza $\sim 10^{-10}$: servono 19 cifre, un `double` ne ha 16.
- Può venire perfino una varianza **negativa**.
- Usate le **due passate** (o sottraete prima ai dati un valore vicino alla media).

Dal periodo all'accelerazione di gravità

Da $T = 2\pi\sqrt{L/g}$ si ricava $g = 4\pi^2 L/T^2$; propagando l'errore, $\sigma_g = 2g\,\sigma_{\bar{T}}/\bar{T}$.

```
// in fondo a stat.c, dopo aver calcolato media ed err
const double pi = 3.141592653589793;
const double L  = 1.000;                  // [m]
double g  = 4.0 * pi * pi * L / (media * media);
double dg = 2.0 * g * err / media;        // propagazione
printf("g     = %.2f +- %.2f m/s^2\n", g, dg);
```

```
$ ./stat < periodi.txt
media = 2.0067 s
sigma = 0.0153 s
T     = 2.0067 +- 0.0015 s
g     = 9.80 +- 0.01 m/s^2
```

- Il risultato si stampa con un numero di cifre adeguato all'**incertezza** (lezione 12): `%.2f` per g, `%.4f` per T.
- Compatibile con $g = 9.81$ m/s² entro l'incertezza.
- **Test** della formula: con $T = 2.0064$ s deve venire $g = 9.807$ m/s².

Media e deviazione standard non dicono tutto: la **forma** della distribuzione si vede con un **istogramma**.

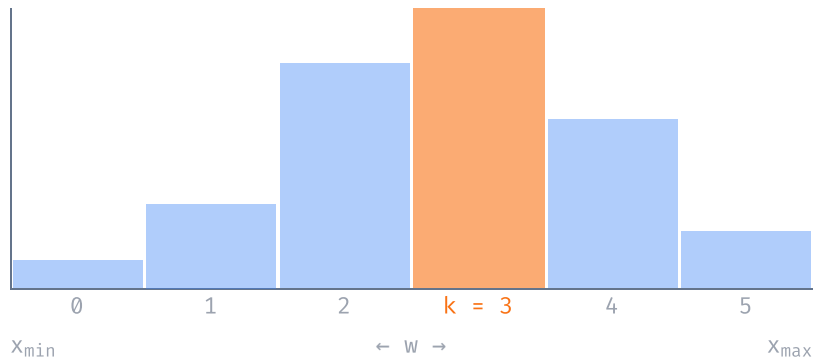
Istogrammi: l'idea

Si divide l'intervallo $[x_{min}, x_{max})$ in `NBIN` intervalli (**bin**) di uguale larghezza e si **conta** quanti dati cadono in ciascuno.

- Larghezza del bin: $w = (x_{max} - x_{min})/\texttt{NBIN}$
- Il dato x cade nel bin di indice

$$k = \left\lfloor \frac{x - x_{min}}{w} \right\rfloor$$

  che in C è un **cast** a `int` (lezione 10).
- I conteggi stanno in un array di **interi** `conteggi[NBIN]`, azzerato all'inizio.
- Per ogni dato: `conteggi[k]++`.



Un dato con $x < x_{min}$ o $x \geq x_{max}$ darebbe un indice **fuori dai limiti** dell'array: va controllato con un `if` !

Istogramma in C

```c
#include <stdio.h>
#define N    100
#define NBIN 10

int main() {
    const double xmin = 1.9595, xmax = 2.0595;  // [s]
    const double w = (xmax - xmin) / NBIN;  // larghezza
    double T[N];
    int conteggi[NBIN] = {0};
    int i, k;

    for (i = 0; i < N; i++) {
        scanf("%lf", &T[i]);
        k = (int) ((T[i] - xmin) / w);
        if (k >= 0 && k < NBIN) {
            conteggi[k]++;
        }
    }
    for (k = 0; k < NBIN; k++) {
        printf("%.4f %d\n", xmin + (k + 0.5) * w,
               conteggi[k]);
    }
    return 0;
}
```

```
$ ./isto < periodi.txt > isto.dat
$ cat isto.dat
1.9645 0
1.9745 3
1.9845 8
1.9945 26
2.0045 22
2.0145 23
2.0245 11
2.0345 4
2.0445 2
2.0545 1
```

- Per ogni bin si stampano il **centro** e il **conteggio**: due colonne pronte per Python.
- Somma dei conteggi: 100 ✓ (nessun dato fuori intervallo).
- Perché `1.9595` e non `1.96`?

Attenzione ai bordi dei bin

```c
double x = 2.010, xmin = 1.96, w = 0.01;
printf("%.17g\n", (x - xmin) / w);
printf("%d\n", (int) ((x - xmin) / w));
```

```
4.9999999999999822
4
```

- Ci aspettiamo $k = 5$: `2.010` sta proprio sul **bordo** tra i bin 4 e 5.
- `2.010`, `1.96`, `0.01` non sono esatti in binario (lezione 4): il risultato è un po' **meno** di 5 e il cast **tronca** a 4.

Bin	`xmin = 1.96`	`xmin = 1.9595`
1.99 – 2.00	26	26
2.00 – 2.01	**26**	22
2.01 – 2.02	**19**	23
2.02 – 2.03	**12**	11

- Con `xmin = 1.96` diversi dati finiscono nel bin **sbagliato**.
- Dati con risoluzione 1 ms: con i bordi a **metà** tra due valori possibili (1.9595, 1.9695, …) nessun dato cade su un bordo.
- La larghezza dei bin sia un **multiplo** della risoluzione.

Istogramma testuale

Con un ciclo **annidato** si stampa una riga di `*` lunga quanto il conteggio.

```c
for (k = 0; k < NBIN; k++) {
    printf("%.4f-%.4f |", xmin + k * w, xmin + (k + 1) * w);
    for (j = 0; j < conteggi[k]; j++) {
        printf("*");
    }
    printf(" %d\n", conteggi[k]);
}
```

- Il ciclo interno viene eseguito `conteggi[k]` volte per **ogni** valore di `k`.
- Utile per un'occhiata veloce dal terminale, senza file né Python.
- Con molti dati conviene **riscalare**: un `*` ogni 10 conteggi.

```
$ ./stelle < periodi.txt
1.9595-1.9695 | 0
1.9695-1.9795 |*** 3
1.9795-1.9895 |******** 8
1.9895-1.9995 |************************** 26
1.9995-2.0095 |********************** 22
2.0095-2.0195 |*********************** 23
2.0195-2.0295 |*********** 11
2.0295-2.0395 |**** 4
2.0395-2.0495 |** 2
2.0495-2.0595 |* 1
```

Istogrammi con Python

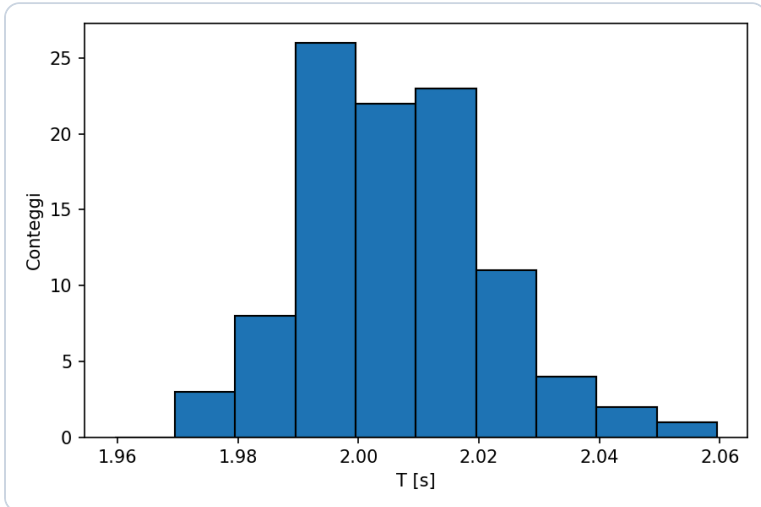
Dai conteggi calcolati in C (`plt.bar`):

```
centro, n = np.loadtxt('isto.dat', unpack=True)
plt.bar(centro, n, width=0.01, edgecolor='black')
```

Direttamente dai dati (`plt.hist`):

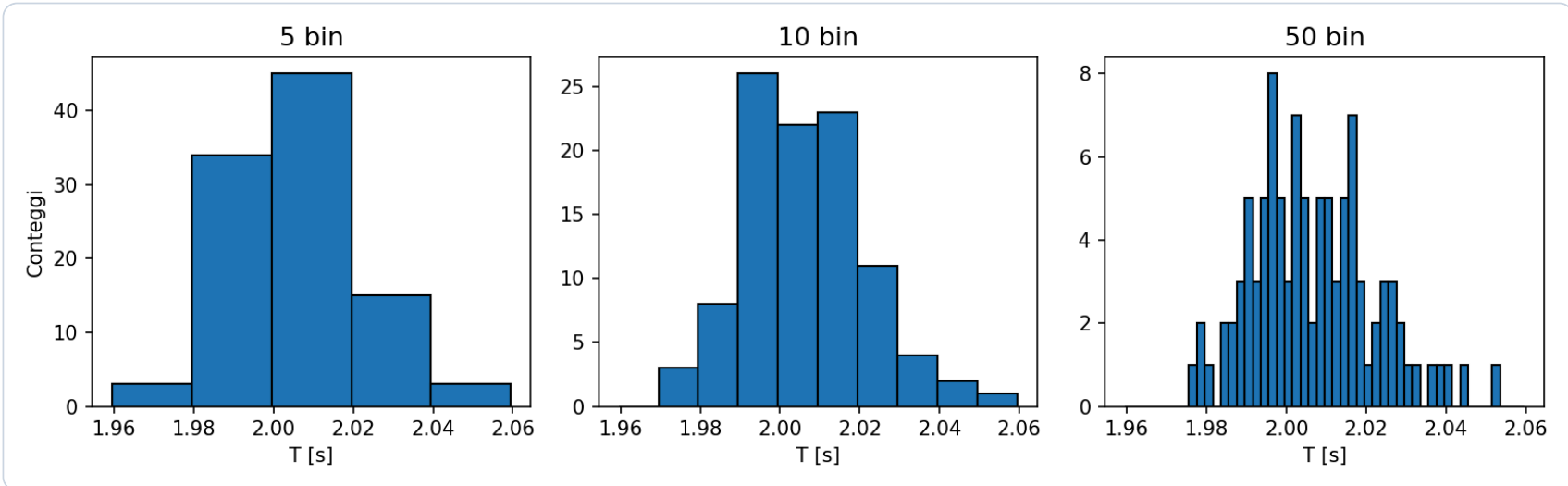
```
import numpy as np
import matplotlib.pyplot as plt

T = np.loadtxt('periodi.txt')
plt.hist(T, bins=10, range=(1.9595, 2.0595),
         edgecolor='black')
plt.xlabel('T [s]')
plt.ylabel('Conteggi')
plt.savefig('isto_periodi.png')
```



- `plt.hist` calcola bin e conteggi da sola;
  `plt.bar` disegna barre di altezza data.
- Con gli stessi bin, i due grafici sono **identici**.

Quanti bin?



Troppo pochi: si perde la forma della distribuzione.

Giusti: si vede la forma "a campana" attorno a $\bar{T}$.

Troppi: pochi conteggi per bin, dominano le **fluttuazioni**.

Regola pratica: circa $\sqrt{N}$ bin (qui $\sqrt{100} = 10$), e mai più fini della risoluzione dello strumento.

Errori tipici

Errore	Cosa succede
Accumulatore non inizializzato (`double somma;`)	Media senza senso: partite da `0.0`
`min` inizializzato a `0`	Con dati positivi il minimo risulta `0` : partite da `T[0]`
Varianza con `^2`	`^` non è la potenza: errore di compilazione con i `double`
Formula a una passata con dati "lontani da zero"	Cancellazione: varianza sbagliata o negativa
`sqrt` senza `-lm`	Errore del linker: `undefined reference to 'sqrt'`